If you want to learn to code from here

> **Note**
>
> *This book taught you to conduct Claude. It didn't teach you to code. Some of you will finish and find yourselves curious about what the artifacts in your `.claude/` folder actually say. This appendix is for you.*

Some readers finish the book and want to keep conducting forever: that's a complete arc; you're done. Some finish and find themselves curious about what the artifacts in their `.claude/` folder actually say.

This appendix is for the second group. Not a coding curriculum: a staged onramp from *"I conduct Claude"* to *"I read what Claude wrote and modify it"* to *"I write small things from scratch."* Three stages, named resources at each.

There is no requirement to leave Stage 1. The book's promise, that English is the only programming language you need to operate an AI staff, is permanent. This appendix exists because some of you will be surprised by your own curiosity. Pick the stage that fits your week, or close the appendix and go back to operating.

Stage 1: Reading what Claude wrote

What this stage is: You stop installing artifacts blind. You open the file Claude generated, read it line by line, ask Claude to explain anything unclear. You don't change anything yet. The shift is from *"Claude wrote it, I trust it"* to *"Claude wrote it, I read it, then I trust it."*

What it is not: Tutorials that teach syntax in the abstract. You learn from the artifacts you already own, in context. The hook in your settings.json is the first thing you read; not a generic JSON tutorial.

The two resources:

- **Andrej Karpathy's *Intro to Large Language Models*** (the one-hour lecture). Watch this not to learn LLMs but to internalize that Claude is a system you can reason about. The same intuition that makes the lecture make sense will make your `.claude/agents/` files make sense. Karpathy's register, narrating

choices while building, is the closest match to this book's conducting-transcript voice. If only one resource lands, make it this one.

- **The companion repo's** `ENGINEER-GUIDE.md` (Elena's reference). Same artifacts as the chapters of this book, written for an engineer. Read the section for any chapter where the artifact intrigued you. The *Chapter 9* morning-brief script section is a good place to start; you've seen it run, you know what it does, and the engineer guide tells you what each line is for.

The signal you're ready for Stage 2: You can read your own `morning-brief.sh` (*Chapter 9*) and explain what each line does to a non-technical friend. Not perfectly; well enough that they'd understand. The "explain to a friend" bar is the right scale, not "explain at a job interview."

Stage 2: Modifying what Claude wrote

What this stage is: You start making small edits yourself. Change a string. Add a `# comment` line. Rename a variable. The deal is: read first, edit second, ask Claude to confirm your change works. Claude is now your editor, not your author.

What it is not: Scaffolding new things from scratch. You are the editor, not the author. The temptation at this stage is to skip ahead: *"I edited a file, I'm a programmer now."* You are not. You are someone learning to read code by editing it, which is different and easier and just as valuable.

The three resources:

- ***Eloquent JavaScript* by Marijn Haverbeke** (free online at `eloquentjavascript.net`). *Chapters 3–6* only. You're learning to read JS, not to write web apps. Stop when the syntax of `morning-brief.sh` and most of your hook scripts no longer feels alien. JS makes sense for this book's readers because most of the artifacts are shell + JSON + the occasional Node-based MCP. (If your scripts are mostly Python (say, because you went deeper on data work) swap in *Python Crash Course* by Eric Matthes. The shape of the recommendation is the same.)
- **Karpathy's *Spelled-out Intro to Neural Networks and Backpropagation*: the Micrograd lecture.** You won't build a neural net. You'll watch someone build software in front of you, narrating choices. *That register* is what you're trying to develop. The lecture is two hours; you don't have to finish in one sitting. The thing to watch for is what he does when he hits an error: that's the move you're learning.
- **Anthropic's published Claude Code architecture docs.** Read these now, not at Stage 1. They make sense once you've edited a hook. Before that, the architecture is abstract; after that, every section maps to a file you've touched.

The signal you're ready for Stage 3: You found a bug in something Claude wrote, fixed it yourself, and didn't break anything else. You felt the satisfaction. The signal is the satisfaction; if a fix didn't feel like *anything*, it was probably trivial enough that Stage 1 covered it. The Stage 2 satisfaction is specific: it's the *"oh, I did that"* moment when an edit lands cleanly.

Stage 3: Writing small things from scratch

What this stage is: You start writing 10–30 line scripts without prompting Claude first. You'll mostly write them poorly. Claude becomes your editor, not your author: you ask it to review what you wrote, find what's

wrong, suggest the cleaner shape. The flip from *Claude writes, you read* to *you write, Claude reads* is the whole arc of this stage.

What it is not: Becoming a software engineer. This stage's destination is *"I can build a small thing on my own when I want to"*, not *"I can interview at Google."* The book has nothing to say about that other journey. Other books do.

The three resources:

- ***Crafting Interpreters* by Robert Nystrom** (free online at `craftinginterpreters.com`). The book that teaches you that programs are just text being interpreted by other programs. Pick this up only if you want to. Skip if not. The reader who finishes this book understands more about what Claude actually *is* than most working developers do, but the path to that understanding is long and optional.
- **Andrej Karpathy's *Let's build GPT* lecture.** For the rare reader who wants to understand the thing they've been operating. Optional. Beautiful. If Karpathy's first two appearances in this appendix landed, the third one will land harder.
- **A small project of your own: pick one.** This is the single best Stage 3 resource. *One project you actually want to ship and 30 minutes a day.* Not "build a SaaS." Not "build the next Claude Code." Build a thing your week needs that you'd otherwise pay for or live without. Maybe Appendix D's bank CSV importer. Maybe a script that does one specific thing for your particular life. Whatever it is, the constraint is that it has to be *yours* and it has to be *small*. Big projects at this stage are graveyards.

The signal you've graduated this appendix: You no longer need this appendix. You know where you are in the journey. You set your own pace. There's no formal credential at the end of Stage 3 because there's no formal credential being offered: you're learning to read and write small programs because you wanted to, not because you were sold a destination.

A closing thought

Karpathy appears three times above because his register (narrating choices while building, refusing to skip the parts that look obvious) is the closest match to this book's conducting-transcript voice. The continuity is the bridge. His specific URLs change; the errata page tracks current ones.

You don't owe anyone a Stage 2 graduation, or a Stage 1 graduation either. For readers who *do* find themselves curious: walk slowly. Karpathy first. The companion engineer guide second. Most people I know who became comfortable reading code did it on the side of a job that didn't require it, with no deadline, with curiosity as the only fuel. That fuel works. *"I have to learn this by Christmas"* doesn't.

Pick the stage that fits your week. Or close the appendix and open the terminal. The staff is upstairs.